

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное
учреждение высшего образования
«Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий и искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторных работ № 6
по дисциплине «Теория графов»

Студент,

группа 5130201/40001

_____ Ови Мд Шамин Ясир

Преподаватель,

_____ Востров Алексей Владимирович

« ____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Математическое описание	5
2.1 Красно-чёрное дерево	5
2.2 Хеш-таблица	6
3 Особенности реализации	8
3.1 Структура RBNode и класс RBTree	8
3.2 rotateLeft и rotateRight	8
3.3 insertFixup	9
3.4 insert (вставка в КЧ-дерево)	9
3.5 searchNode (поиск)	10
3.6 deleteFixup	10
3.7 remove (удаление из КЧ-дерева)	11
3.8 printTreeHelper (визуализация дерева)	11
3.9 Хеш-функция	12
3.10 insert, search, remove (хеш-таблица)	12
3.11 printStats (статистика хеш-таблицы)	13
3.12 Обработка текста на русском языке (UTF-8)	14
4 Результаты работы программы	15
4.1 Главное меню	15
4.2 Загрузка текста (пункт 1)	15
4.3 Добавление слова в КЧ-дерево (пункт 3)	16
4.4 Поиск слова в КЧ-дереве (пункт 4)	19
4.5 Удаление слова из КЧ-дерева (пункт 5)	20
4.6 Печать словаря КЧ-дерева (пункт 6)	21
4.7 Структура КЧ-дерева (пункт 7)	23
4.8 Очистка КЧ-дерева (пункт 8)	26
4.9 Добавление слова в хеш-таблицу (пункт 9)	26
4.10 Поиск слова в хеш-таблице (пункт 10)	27
4.11 Удаление слова из хеш-таблицы (пункт 11)	27
4.12 Печать хеш-таблицы (пункт 12)	27
4.13 Статистика хеш-таблицы (пункт 13)	28
4.14 Очистка хеш-таблицы (пункт 14)	29
4.15 Выход из программы (пункт 0)	29
Заключение	30
Список использованной литературы	31

Введение

В лабораторной работе требуется для выбранного текста на русском языке реализовать два словаря с использованием различных структур данных: хеш-таблицы и красно-чёрного дерева.

Первый словарь реализован на основе хеш-таблицы с методом цепочек для разрешения коллизий. Хеш-функция выбрана полиномиальная (нетривиальная). Для него реализованы операции добавления, удаления, поиска, полной очистки и загрузки/дополнения словаря из текстового файла.

Второй словарь построен на основе красно-чёрного дерева. Реализованы функции добавления, удаления и поиска слова, не используя готовые структуры данных (map, set и т. д.). Добавлены функции полной очистки словаря и загрузки/дополнения словаря из текстового файла.

В качестве исходного текста используется стихотворение А. С. Пушкина «Я вас любил», закодированное в формате UTF-8.

Лабораторная работа была выполнена на языке программирования C++, в среде разработки CLion.

1 Постановка задачи

Для произвольного текста на русском языке необходимо:

1. Реализовать структуру данных — хеш-таблицу, для которой характерны операции: добавления, удаления и поиска. На её основе реализовать словарь. Хеш-функцию выбрать произвольную нетривиальную. Добавить функцию полной очистки словаря и загрузки/дополнения словаря из текстового файла.
2. Для этого же текста на русском языке построить словарь на основе красно-чёрных деревьев. Реализовать функции добавления, удаления и поиска слова, не используя готовые структуры данных. Добавить функцию полной очистки словаря и загрузки/дополнения словаря из текстового файла.

Программа должна обеспечивать:

- корректную работу с русским текстом в кодировке UTF-8;
- визуализацию состояния дерева (структура, цвета узлов, NIL-листья);
- отображение статистики хеш-таблицы (количество коллизий, коэффициент загрузки, максимальная длина цепочки);
- устойчивую работу программы после последовательных операций вставки и удаления.

2 Математическое описание

2.1 Красно-чёрное дерево

Красно-чёрное дерево — разновидность бинарного дерева поиска, баланс которого поддерживается за счёт окрашивания узлов в два цвета по определённым правилам:

- Каждый узел окрашен либо в красный, либо в чёрный цвет;
- Корень дерева всегда чёрный;
- Листьями считаются NIL-узлы («виртуальные» узлы), которые всегда чёрные;
- Если узел красный, то оба его потомка должны быть чёрными (нет двух красных подряд);
- На всех путях от корня до листьев количество чёрных узлов одинаково (чёрная высота).

Для данной работы реализованы операции вставки и удаления узла, поиска значения, считывания текста из файла, визуализации дерева в консоли и очистки словаря.

Поиск в красно-чёрном дереве совпадает с поиском в обычном двоичном дереве поиска. Начиная с корня, сравниваем искомый ключ с ключом текущего узла и переходим в левое или правое поддерево. Сложность поиска — $O(\log n)$.

При вставке и удалении узлов происходит балансировка для сохранения свойств красно-чёрного дерева.

Вставка:

- Новый узел всегда вставляется как красный.
- Если возникает нарушение свойств (два красных подряд), выполняется балансировка.
- Балансировка при вставке включает 3 основных случая (и 3 зеркальных):
 1. Дядя красный — перекрашиваем родителя, дядю и деда, продвигаемся вверх;
 2. Дядя чёрный, узел — правый ребёнок (треугольник) — левый поворот, сводим к случаю 3;
 3. Дядя чёрный, узел — левый ребёнок (линия) — правый поворот деда, перекраска.

Удаление:

- Если удаляемый узел имеет менее двух потомков — заменяем его потомком.
- Если у удаляемого узла два потомка — находим его преемника (минимум в правом поддереве), копируем значение и удаляем преемника.
- Если удалён чёрный узел, нарушается чёрная высота — вызывается deleteFixup с 4 случаями (и 4 зеркальными):
 1. Брат красный — перекраска и поворот;
 2. Брат чёрный, оба ребёнка чёрные — перекрашиваем брата, продвигаемся вверх;
 3. Брат чёрный, дальний ребёнок чёрный — поворот брата, сводим к случаю 4;
 4. Брат чёрный, дальний ребёнок красный — финальный поворот и перекраска.

Вращения:

Левое и правое вращения изменяют структуру дерева, сохраняя порядок элементов. Левое вращение вокруг узла x перемещает правого потомка y на место x , а x становится левым потомком y . Правое вращение — зеркальная операция.

Результаты балансировки:

- Высота дерева остаётся $O(\log n)$ даже при последовательных вставках и удалениях;
- Сохраняется свойство упорядоченности: левое поддерево $<$ родитель $<$ правое поддерево;
- Гарантируется быстрый поиск, вставка и удаление за $O(\log n)$.

2.2 Хеш-таблица

Хеш-таблица — структура данных, позволяющая хранить пары «ключ-значение» с быстрым доступом по ключу. Хеш-функция вычисляет индекс в массиве (корзину) по ключу.

Хеш-функция:

В данной работе используется полиномиальная хеш-функция:

$$H(s) = \left(\sum_{i=0}^{n-1} s_i \cdot 31^{n-1-i} \right) \bmod N$$

где s_i — код i -го байта слова, n — длина слова, 31 — простое число (основание), $N = 127$ — размер хеш-таблицы (простое число).

Почему выбрано основание 31:

1. 31 — простое число, что обеспечивает лучшее распределение хеш-значений и меньше коллизий;
2. $31 = 2^5 - 1$, поэтому умножение на 31 компилятор оптимизирует в $(\text{hash} \ll 5) - \text{hash}$ (быстрее обычного умножения);
3. Достаточно малое для избежания переполнения, но достаточно большое для разброса значений;
4. Нечётные простые числа работают лучше чётных, так как умножение на чётное число сдвигает биты влево и теряет информацию.

Почему размер таблицы 127: это простое число, и $\text{gcd}(31, 127) = 1$, что гарантирует равномерное распределение по корзинам без кластеризации.

Качество хеш-функции:

Полиномиальная хеш-функция использует *все* символы слова, а не только первый символ (что было бы тривиальным подходом). Для 36 уникальных слов из тестового текста получено всего 4 коллизии при 127 корзинах (коэффициент загрузки 0.28), что свидетельствует о хорошем качестве функции.

Разрешение коллизий — метод цепочек:

Каждая ячейка таблицы содержит указатель на связный список элементов с одинаковым хеш-значением. При коллизии новый элемент добавляется в начало списка.

Основные операции:

- **Вставка.** Вычисляется хеш-индекс, проверяется наличие слова в цепочке. Если слово найдено — увеличивается счётчик. Если не найдено — создаётся новый узел. Сложность: $O(1)$ в среднем, $O(n)$ в худшем.

- **Поиск.** Вычисляется хеш-индекс, обходится цепочка до совпадения или конца. Сложность: $O(1)$ в среднем.
- **Удаление.** Аналогично поиску, но при нахождении узла он удаляется из цепочки.

Таблица 1. Пример хеш-таблицы (фрагмент)

Ключ	Количество	Хеш
как	1	2
любовь	1	8
я	4	18
вас	5	33
так	2	69
не	3	115
любил	3	108
быть	2	126

3 Особенности реализации

Программа разбита на файлы: `rbtree.h/cpp` (красно-чёрное дерево), `hashtable.h/cpp` (хеш-таблица), `textutils.h/cpp` (обработка текста) и `main.cpp` (меню). Все структуры данных реализованы вручную, без использования готовых контейнеров (`map`, `set`, `unordered_map`).

3.1 Структура RBNode и класс RBTree

Структура `RBNode` содержит слово, счётчик вхождений, цвет (`RED/BLACK`) и указатели на левого, правого потомка и родителя. Используется узел-страж `NIL` (всегда чёрный). Код представлен в листинге 1.

Ввод: при создании узла задаются слово, счётчик, цвет и указатели на соседние узлы.

Вывод: структура хранит один элемент словаря и связи, необходимые для работы красно-чёрного дерева.

```
1 enum Color { RED, BLACK };
2
3 struct RBNode {
4     string word;
5     int count;
6     Color color;
7     RBNode* left;
8     RBNode* right;
9     RBNode* parent;
10 };
```

Листинг 1. Структура `RBNode`

3.2 rotateLeft и rotateRight

Левый поворот вокруг узла x перемещает его правого потомка y на место x . Правый поворот — зеркальная операция. Код представлен в листинге 2.

Ввод: указатель на узел x , вокруг которого выполняется поворот.

Вывод: изменяются связи между узлами дерева, при этом порядок ключей сохраняется.

```
1 void RBTree::rotateLeft(RBNode* x) {
2     RBNode* y = x->right;
3     x->right = y->left;
4     if (y->left != NIL) y->left->parent = x;
5     y->parent = x->parent;
6     if (x->parent == nullptr) root = y;
7     else if (x == x->parent->left) x->parent->left = y;
8     else x->parent->right = y;
9     y->left = x;
10    x->parent = y;
11 }
12
13 void RBTree::rotateRight(RBNode* x) {
14     RBNode* y = x->left;
15     x->left = y->right;
16     if (y->right != NIL) y->right->parent = x;
17     y->parent = x->parent;
18     if (x->parent == nullptr) root = y;
19     else if (x == x->parent->right) x->parent->right = y;
20     else x->parent->left = y;
```



```

21     y->right = x;
22     x->parent = y;
23 }

```

Листинг 2. Функции rotateLeft и rotateRight

3.3 insertFixup

Восстанавливает свойства КЧ-дерева после вставки. Обрабатывает 3 случая и их зеркальные отражения. Код представлен в листинге 3.

Ввод: указатель на новый красный узел `z`, вставленный по правилам бинарного дерева поиска.

Вывод: дерево перебалансируется с помощью перекраски и поворотов, корень становится чёрным.

```

1 void RBTREE::insertFixup(RBNode* z) {
2     while (z->parent != nullptr && z->parent->color == RED) {
3         if (z->parent == z->parent->parent->left) {
4             RBNode* uncle = z->parent->parent->right;
5             if (uncle->color == RED) {
6                 z->parent->color = BLACK;
7                 uncle->color = BLACK;
8                 z->parent->parent->color = RED;
9                 z = z->parent->parent;
10            } else {
11                if (z == z->parent->right) {
12                    z = z->parent;
13                    rotateLeft(z);
14                }
15                z->parent->color = BLACK;
16                z->parent->parent->color = RED;
17                rotateRight(z->parent->parent);
18            }
19        } else { /* Mirror cases */ }
20    }
21    root->color = BLACK;
22 }

```

Листинг 3. Функция insertFixup

3.4 insert (вставка в КЧ-дерево)

Если слово уже существует — увеличивает счётчик. Иначе создаёт новый красный узел, находит его позицию по правилам BST и вызывает insertFixup. Код представлен в листинге 4.

Ввод: слово `word`, приведённое к нижнему регистру.

Вывод: слово добавляется в дерево или увеличивается его счётчик; при необходимости выполняется балансировка.

```

1 void RBTREE::insert(const string& word) {
2     RBNode* existing = searchNode(word);
3     if (existing != NIL) { existing->count++; return; }
4
5     RBNode* z = new RBNode();
6     z->word = word; z->count = 1; z->color = RED;
7     z->left = NIL; z->right = NIL; z->parent = nullptr;
8

```

```

9   RBNode* y = nullptr;
10  RBNode* x = root;
11  while (x != NIL) {
12      y = x;
13      if (word < x->word) x = x->left;
14      else x = x->right;
15  }
16  z->parent = y;
17  if (y == nullptr) root = z;
18  else if (word < y->word) y->left = z;
19  else y->right = z;
20
21  if (z->parent == nullptr) { z->color = BLACK; return; }
22  if (z->parent->parent == nullptr) return;
23  insertFixup(z);
24 }

```

Листинг 4. Функция insert

3.5 searchNode (поиск)

Стандартный поиск в BST: сравниваем ключ с текущим узлом и переходим влево или вправо. Код представлен в листинге 5.

Ввод: искомое слово word.

Вывод: указатель на найденный узел или NIL, если слово отсутствует.

```

1  RBNode* RBTREE::searchNode(const string& word) {
2      RBNode* current = root;
3      while (current != NIL) {
4          if (word == current->word) return current;
5          else if (word < current->word) current = current->left;
6          else current = current->right;
7      }
8      return NIL;
9  }

```

Листинг 5. Функция searchNode

3.6 deleteFixup

Восстанавливает свойства КЧ-дерева после удаления чёрного узла. 4 случая и 4 зеркальных. Код представлен в листинге 6.

Ввод: узел x, который занял место удалённого чёрного узла.

Вывод: свойства красно-чёрного дерева восстанавливаются с помощью перекраски и поворотов.

```

1  void RBTREE::deleteFixup(RBNode* x) {
2      while (x != root && x->color == BLACK) {
3          if (x == x->parent->left) {
4              RBNode* w = x->parent->right;
5              if (w->color == RED) { /* Case 1 */
6                  w->color = BLACK; x->parent->color = RED;
7                  rotateLeft(x->parent); w = x->parent->right;
8              }
9              if (w->left->color == BLACK && w->right->color == BLACK) {
10                 w->color = RED; x = x->parent; /* Case 2 */
11             } else {
12                 if (w->right->color == BLACK) { /* Case 3 */

```

```

13         w->left->color = BLACK; w->color = RED;
14         rotateRight(w); w = x->parent->right;
15     }
16     /* Case 4 */
17     w->color = x->parent->color;
18     x->parent->color = BLACK;
19     w->right->color = BLACK;
20     rotateLeft(x->parent); x = root;
21 }
22 } else { /* Mirror cases */ }
23 }
24 x->color = BLACK;
25 }

```

Листинг 6. Функция deleteFixup (левая ветвь)

3.7 remove (удаление из КЧ-дерева)

Обрабатывает три случая: узел без левого потомка, без правого, с двумя потомками (через преемника). При удалении чёрного узла вызывается deleteFixup. Код представлен в листинге 7.

Ввод: слово `word`, которое нужно удалить из дерева.

Вывод: возвращается `true` при успешном удалении и `false`, если слово не найдено; дерево сохраняет свойства КЧ-дерева.

```

1 bool RBTREE::remove(const string& word) {
2     RBNode* z = searchNode(word);
3     if (z == NIL) return false;
4     RBNode* y = z; RBNode* x;
5     Color yOriginalColor = y->color;
6     if (z->left == NIL) {
7         x = z->right; transplant(z, z->right);
8     } else if (z->right == NIL) {
9         x = z->left; transplant(z, z->left);
10    } else {
11        y = treeMinimum(z->right);
12        yOriginalColor = y->color; x = y->right;
13        if (y->parent == z) { x->parent = y; }
14        else {
15            transplant(y, y->right);
16            y->right = z->right; y->right->parent = y;
17        }
18        transplant(z, y);
19        y->left = z->left; y->left->parent = y;
20        y->color = z->color;
21    }
22    delete z;
23    if (yOriginalColor == BLACK) deleteFixup(x);
24    return true;
25 }

```

Листинг 7. Функция remove

3.8 printTreeHelper (визуализация дерева)

Рекурсивно выводит дерево боковым способом: правое поддерево сверху, левое снизу. Узлы отмечены [R] или [B]. NIL-узлы отображаются как чёрные листья. Код представлен в листинге 8.

Ввод: текущий узел `node` и величина отступа `indent`.

Вывод: на экран печатается фрагмент структуры дерева с цветами узлов и NIL-листьями.

```
1 void RBTREE::printTreeHelper(RBNode* node, int indent) {
2     if (node == NIL) {
3         cout << string(indent, ' ') << "[B] NIL" << endl;
4         return;
5     }
6     printTreeHelper(node->right, indent + 6);
7     cout << string(indent, ' ')
8         << (node->color == RED ? "[R] " : "[B] ")
9         << node->word << "(" << node->count << ")" << endl;
10    printTreeHelper(node->left, indent + 6);
11 }
```

Листинг 8. Визуализация дерева

3.9 Хеш-функция

Полиномиальная хеш-функция с основанием 31. Использует все символы слова. Код представлен в листинге 9.

Ввод: слово `word`.

Вывод: индекс корзины хеш-таблицы в диапазоне от 0 до 126.

```
1 // Why 31: prime, 2^5-1 (fast), good spread, same as Java
2 // Why TABLE_SIZE=127: prime, gcd(31,127)=1 -> no clusters
3 unsigned int HashTable::hashFunction(const string& word) {
4     unsigned int hash = 0;
5     for (size_t i = 0; i < word.length(); i++) {
6         hash = hash * 31 + (unsigned char)word[i];
7     }
8     return hash % TABLE_SIZE;
9 }
```

Листинг 9. Функция hashFunction

3.10 insert, search, remove (хеш-таблица)

Вставка: вычисляется хеш, проверяется цепочка; если слово найдено — увеличивается счётчик, иначе создаётся новый узел в начале цепочки.

Поиск: вычисляется хеш, обходится цепочка до совпадения.

Удаление: аналогично поиску, но при нахождении узел удаляется с корректировкой указателей.

Код представлен в листинге 10.

Ввод: слово `word` для добавления, поиска или удаления.

Вывод: при вставке слово добавляется или увеличивается его счётчик; при поиске возвращается `true/false`; при удалении найденный узел удаляется из цепочки.

```
1 void HashTable::insert(const string& word) {
2     unsigned int index = hashFunction(word);
3     HashNode* current = table[index];
4     while (current != nullptr) {
5         if (current->word == word) { current->count++; return; }
6         current = current->next;
7     }
8     HashNode* newNode = new HashNode();
```

```

9     newNode->word = word; newNode->count = 1;
10    newNode->next = table[index];
11    table[index] = newNode;
12 }
13
14 bool HashTable::search(const string& word) {
15     unsigned int index = hashFunction(word);
16     HashNode* current = table[index];
17     while (current != nullptr) {
18         if (current->word == word) return true;
19         current = current->next;
20     }
21     return false;
22 }
23
24 bool HashTable::remove(const string& word) {
25     unsigned int index = hashFunction(word);
26     HashNode* current = table[index];
27     HashNode* prev = nullptr;
28     while (current != nullptr) {
29         if (current->word == word) {
30             if (prev == nullptr) table[index] = current->next;
31             else prev->next = current->next;
32             delete current; return true;
33         }
34         prev = current; current = current->next;
35     }
36     return false;
37 }

```

Листинг 10. Вставка, поиск и удаление в хеш-таблице

3.11 printStats (статистика хеш-таблицы)

Выводит размер таблицы, количество занятых корзин, число коллизий, максимальную длину цепочки и коэффициент загрузки. Код представлен в листинге 11.

Ввод: текущее состояние хеш-таблицы.

Вывод: на экран выводятся размер таблицы, число занятых корзин, коэффициент загрузки, число коллизий и максимальная длина цепочки.

```

1 void HashTable::printStats() {
2     int usedBuckets = 0, totalWords = 0;
3     int maxChain = 0, collisions = 0;
4     for (int i = 0; i < TABLE_SIZE; i++) {
5         int chainLen = 0;
6         HashNode* current = table[i];
7         while (current != nullptr) { chainLen++; current = current->next; }
8         if (chainLen > 0) usedBuckets++;
9         if (chainLen > 1) collisions += (chainLen - 1);
10        if (chainLen > maxChain) maxChain = chainLen;
11        totalWords += chainLen;
12    }
13    cout << "Table size: " << TABLE_SIZE << endl;
14    cout << "Used buckets: " << usedBuckets << "/" << TABLE_SIZE << endl;
15    cout << "Load factor: " << (double)totalWords / TABLE_SIZE << endl;
16    cout << "Total collisions: " << collisions << endl;
17 }

```

Листинг 11. Функция printStats

3.12 Обработка текста на русском языке (UTF-8)

Для корректной обработки русского текста реализованы вспомогательные функции: `isRussianLetter` (проверка русской буквы), `isAllRussian` (валидация слова), `toLowerRussian` (приведение к нижнему регистру с учётом двухбайтовых символов кириллицы) и `processText` (извлечение слов из текста и вставка в оба словаря). Код представлен в листинге 12.

Ввод: строка UTF-8 с русским текстом, позиция проверяемого символа и ссылки на оба словаря.

Вывод: русские слова выделяются из текста, приводятся к нижнему регистру и добавляются в красно-чёрное дерево и хеш-таблицу.

```
1 bool isRussianLetter(const string& text, size_t i) {
2     if (i >= text.size()) return false;
3     unsigned char c = text[i];
4     if (c == 0xD0 && i + 1 < text.size()) {
5         unsigned char next = text[i + 1];
6         if ((next >= 0x90 && next <= 0xBF) || next == 0x81) return true;
7     }
8     if (c == 0xD1 && i + 1 < text.size()) {
9         unsigned char next = text[i + 1];
10        if ((next >= 0x80 && next <= 0x8F) || next == 0x91) return true;
11    }
12    return false;
13 }
14
15 void processText(const string& text, RBTree& tree, HashTable& ht) {
16     string word;
17     size_t i = 0;
18     while (i < text.size()) {
19         if (isRussianLetter(text, i)) {
20             word += text[i]; word += text[i + 1]; i += 2;
21         } else {
22             if (!word.empty()) {
23                 string lower = toLowerRussian(word);
24                 tree.insert(lower); ht.insert(lower);
25                 word.clear();
26             }
27             i++;
28         }
29     }
30     if (!word.empty()) {
31         string lower = toLowerRussian(word);
32         tree.insert(lower); ht.insert(lower);
33     }
34 }
```

Листинг 12. Обработка русского текста

4 Результаты работы программы

4.1 Главное меню

На рис. 1 представлено главное меню программы с 14 доступными операциями для обоих словарей.

```
=====
Lab 6: Dictionary (RB-Tree + Hash Table)
=====

--- Main Menu ---
1. Load default text (Пушкин - Я вас любил)
2. Load text from file
--- Red-Black Tree ---
3. Add word (RB-Tree)
4. Search word (RB-Tree)
5. Delete word (RB-Tree)
6. Print dictionary sorted (RB-Tree)
7. Print tree structure (RB-Tree)
8. Clear dictionary (RB-Tree)
--- Hash Table ---
9. Add word (Hash Table)
10. Search word (Hash Table)
11. Delete word (Hash Table)
12. Print dictionary (Hash Table)
13. Print statistics (Hash Table)
14. Clear dictionary (Hash Table)
---
0. Exit
Choice:
```

Рис. 1. Главное меню программы

4.2 Загрузка текста (пункт 1)

На рис. 2 показана загрузка стихотворения А. С. Пушкина «Я вас любил» в оба словаря одновременно.

```
Choice:1
1
Default text loaded (Пушкин - Я вас любил).

--- Main Menu ---
1. Load default text (Пушкин - Я вас любил)
2. Load text from file
--- Red-Black Tree ---
3. Add word (RB-Tree)
4. Search word (RB-Tree)
5. Delete word (RB-Tree)
6. Print dictionary sorted (RB-Tree)
7. Print tree structure (RB-Tree)
8. Clear dictionary (RB-Tree)
--- Hash Table ---
9. Add word (Hash Table)
10. Search word (Hash Table)
11. Delete word (Hash Table)
12. Print dictionary (Hash Table)
13. Print statistics (Hash Table)
14. Clear dictionary (Hash Table)
---
0. Exit
Choice:
```

Рис. 2. Загрузка текста по умолчанию

4.3 Добавление слова в КЧ-дерево (пункт 3)

На рис. 3–5 показано добавление слова «Ови» в КЧ-дерево и визуализация дерева после вставки. Видно, что слово приведено к нижнему регистру и вставлено как красный узел.


```

Choice:3
3
Enter word to add (Russian only):Ови
Ови
Word "ови" added to RB-Tree.

Tree after insertion:
=== RB-Tree Structure ===
(read sideways: right on top, left on bottom)
[B] = Black, [R] = Red
---
                [B] NIL
            [B] я(4)
                [B] NIL
                [R] хочу(1)
                [B] NIL
        [B] угасла(1)
            [B] NIL
            [B] тревожит(1)
                [B] NIL
                [R] томим(1)
                [B] NIL
[B] то(2)
                [B] NIL
                [R] так(2)
                [B] NIL
                [B] совсем(1)
                [B] NIL
                [R] робостью(1)
                [B] NIL
                [B] ревностью(1)
                [B] NIL
[B] пусть(1)

```

Рис. 3. Добавление слова в КЧ-дерево (верхняя часть дерева)

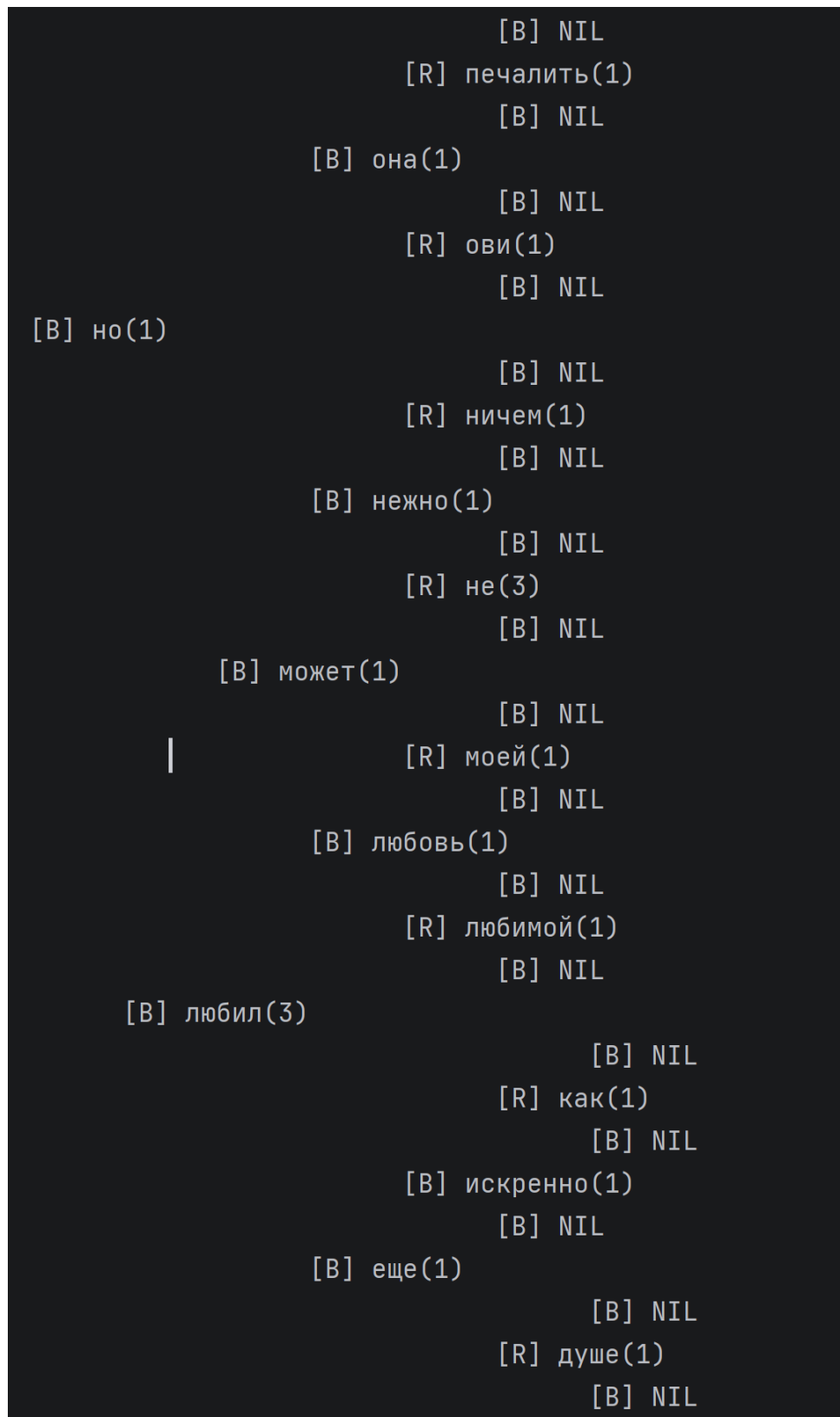


Рис. 4. Дерево после вставки (средняя часть)

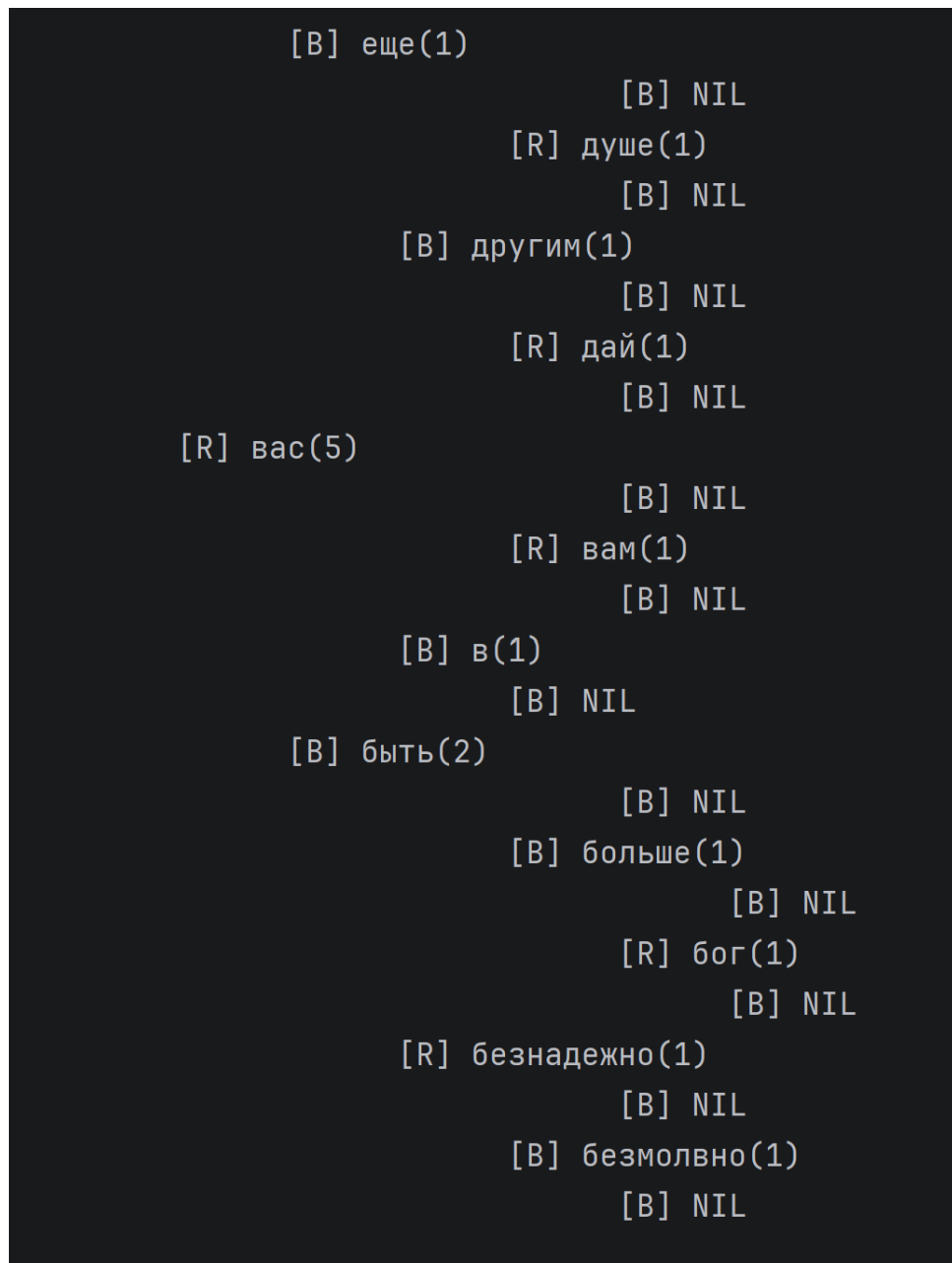


Рис. 5. Дерево после вставки (нижняя часть)

4.4 Поиск слова в КЧ-дереве (пункт 4)

На рис. 6 показан поиск слова «я» в КЧ-дереве. Слово найдено с количеством вхождений 4.

```

Choice:4
4
Enter word to search (Russian only):я
я
Found: "я" (count: 4)

--- Main Menu ---
1. Load default text (Пушкин - Я вас любил)
2. Load text from file
--- Red-Black Tree ---
3. Add word (RB-Tree)
4. Search word (RB-Tree)
5. Delete word (RB-Tree)
6. Print dictionary sorted (RB-Tree)
7. Print tree structure (RB-Tree)
8. Clear dictionary (RB-Tree)
--- Hash Table ---
9. Add word (Hash Table)
10. Search word (Hash Table)
11. Delete word (Hash Table)
12. Print dictionary (Hash Table)
13. Print statistics (Hash Table)
14. Clear dictionary (Hash Table)
---
0. Exit
Choice:|

```

Рис. 6. Поиск слова в КЧ-дереве

4.5 Удаление слова из КЧ-деревя (пункт 5)

На рис. 7 и 8 показано состояние дерева до и после удаления узла. После удаления дерево перебалансируется и сохраняет все свойства КЧ-деревя.

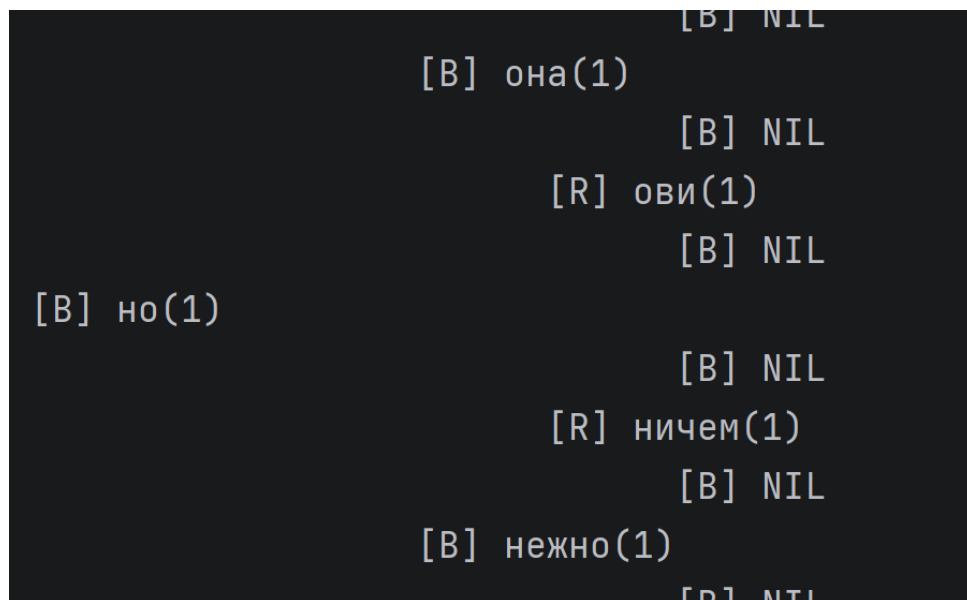


Рис. 7. Фрагмент дерева ДО удаления

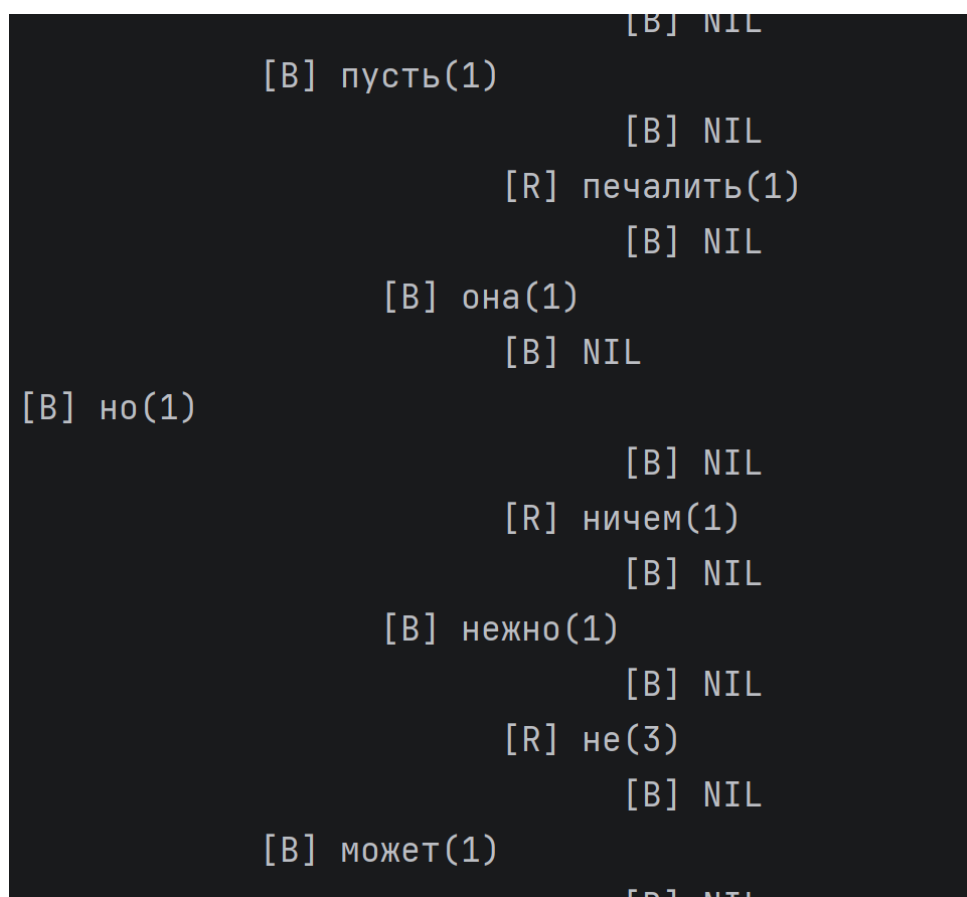


Рис. 8. Фрагмент дерева ПОСЛЕ удаления

4.6 Печать словаря КЧ-дерева (пункт 6)

На рис. 9 и 10 показан отсортированный словарь (обход дерева in-order). Всего 36 уникальных слов.

```
Choice:6
6
=== RB-Tree Dictionary (sorted) ===
безмолвно : 1
безнадежно : 1
бог : 1
больше : 1
быть : 2
в : 1
вам : 1
вас : 5
дай : 1
другим : 1
душе : 1
еще : 1
искренно : 1
как : 1
любил : 3
любимой : 1
любовь : 1
моей : 1
может : 1
не : 3
нежно : 1
ничем : 1
но : 1
она : 1
печалить : 1
пусть : 1
ревностью : 1
```

Рис. 9. Словарь КЧ-дерева, отсортированный (начало)

```
пусть : 1
ревностью : 1
робостью : 1
совсем : 1
так : 2
то : 2
томим : 1
тревожит : 1
угасла : 1
хочу : 1
я : 4
Total words: 36
```

Рис. 10. Словарь КЧ-дерева, отсортированный (окончание)

4.7 Структура КЧ-дерева (пункт 7)

На рис. 11–13 показана визуализация структуры дерева. Дерево выводится боковым способом: правое поддерево сверху, левое — снизу. Узлы отмечены [R] (красный) и [B] (чёрный), NIL-листья отображаются как чёрные.

```
Choice:7
7
=== RB-Tree Structure ===
(read sideways: right on top, left on bottom)
[B] = Black, [R] = Red
---
          [B] NIL
        [B] я(4)
              [B] NIL
            [R] хочу(1)
              [B] NIL
          [B] угасла(1)
            [B] NIL
          [B] тревожит(1)
              [B] NIL
            [R] томим(1)
              [B] NIL
[B] то(2)
              [B] NIL
            [R] так(2)
              [B] NIL
          [B] совсем(1)
            [B] NIL
          [R] робостью(1)
              [B] NIL
            [B] ревностью(1)
              [B] NIL
          [B] пусть(1)
              [B] NIL
            [R] печалить(1)
              [B] NIL
```

Рис. 11. Структура КЧ-дерева (верхняя часть)

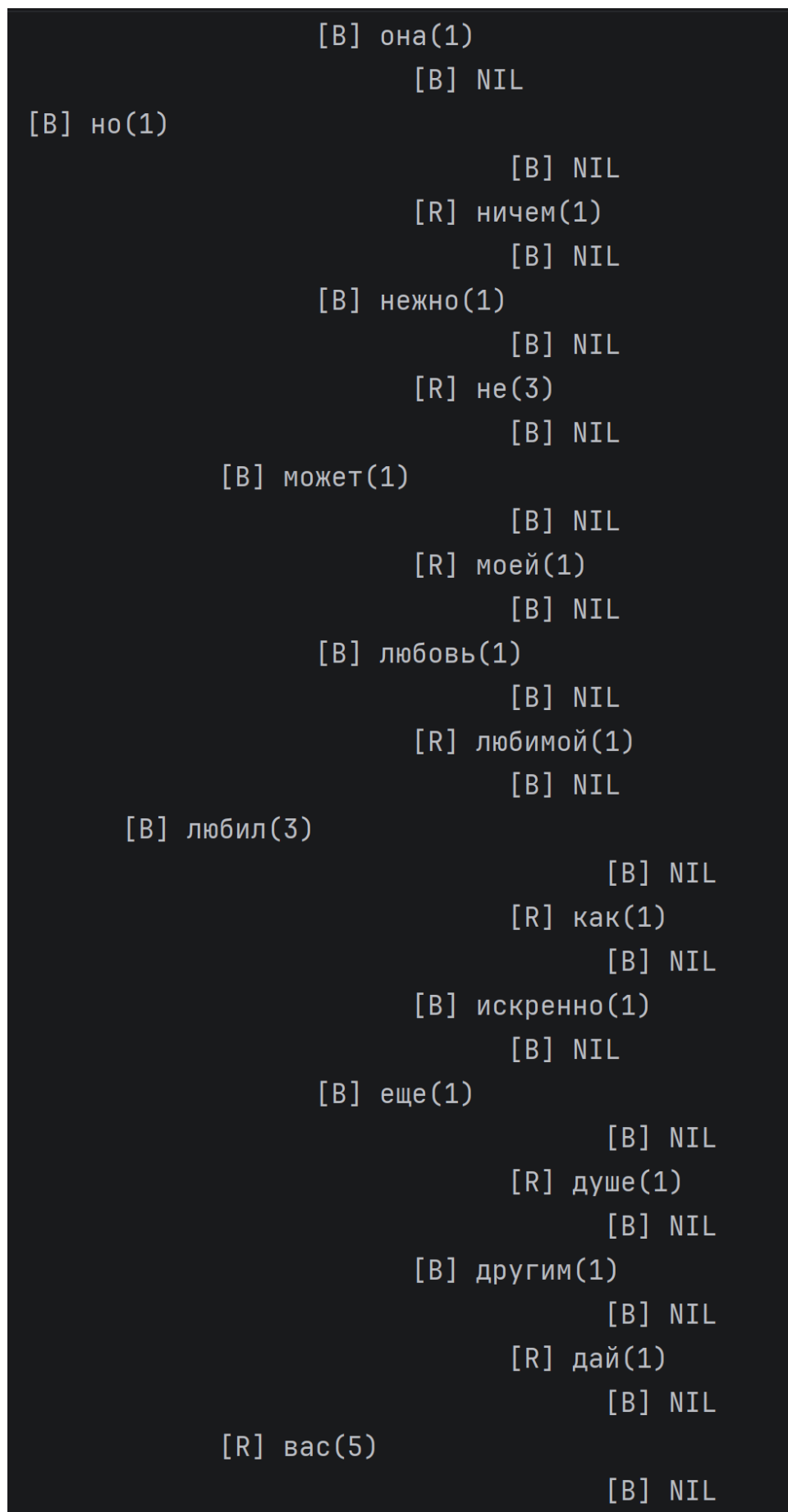


Рис. 12. Структура КЧ-дерева (средняя часть)

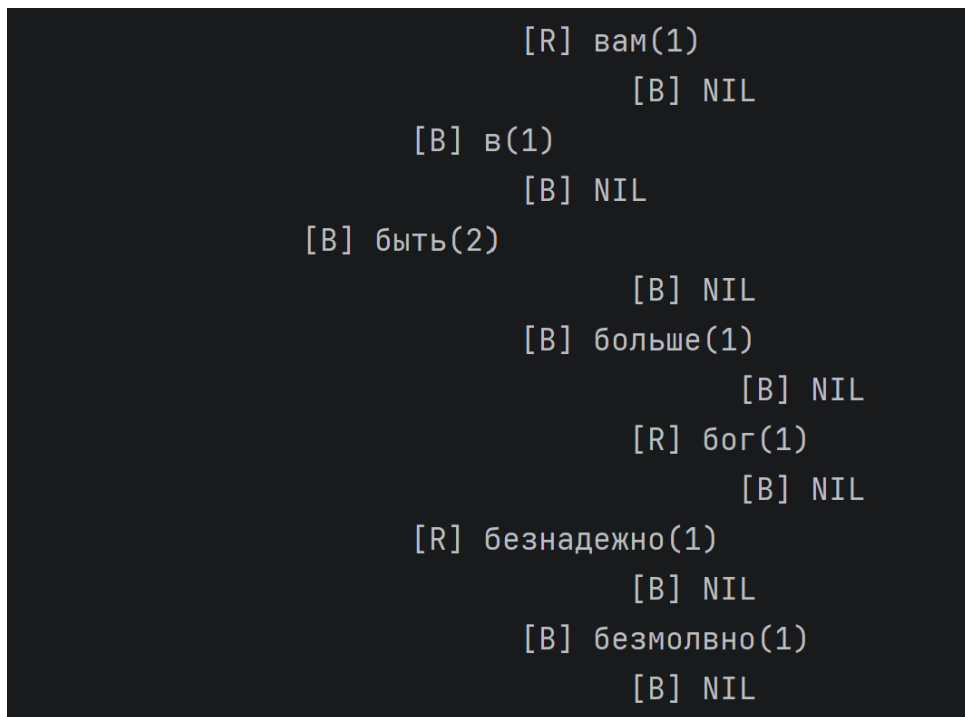


Рис. 13. Структура КЧ-дерева (нижняя часть)

4.8 Очистка КЧ-дерева (пункт 8)

На рис. 14 показана очистка словаря КЧ-дерева.

```
Choice:8
8
RB-Tree dictionary cleared.
```

Рис. 14. Очистка словаря КЧ-дерева

4.9 Добавление слова в хеш-таблицу (пункт 9)

На рис. 15 показано добавление слова «Ови» в хеш-таблицу. Слово приводится к нижнему регистру.

```
Choice:9
9
Enter word to add (Russian only):Ови
Ови
Word "ови" added to Hash Table.
```

Рис. 15. Добавление слова в хеш-таблицу

4.10 Поиск слова в хеш-таблице (пункт 10)

На рис. 16 показан поиск слова «я» в хеш-таблице. Выводится количество вхождений и номер корзины (bucket 18).

```
Choice:10
10
Enter word to search (Russian only):Я
Я
Found: "я" (count: 8, bucket: 18)
```

Рис. 16. Поиск слова в хеш-таблице

4.11 Удаление слова из хеш-таблицы (пункт 11)

На рис. 17 показано удаление слова «я» из хеш-таблицы (корзина 18).

```
Choice:11
11
Enter word to delete (Russian only):Я
Я
Deleted "я" from bucket 18
```

Рис. 17. Удаление слова из хеш-таблицы

4.12 Печать хеш-таблицы (пункт 12)

На рис. 18 и 19 показано содержимое хеш-таблицы. Видны коллизии, отмеченные пометкой [collision] (например, корзины 17, 44, 80, 124). Всего 36 уникальных слов.

```

Choice:12
12
=== Hash Table Dictionary ===
[2] как(2)
[3] хочу(2)
[8] любовь(2)
[10] пусть(2)
[11] может(2)
[12] дай(2)
[13] то(4)
[17] ревностью(2) -> моей(2) [collision]
[22] в(2)
[31] любимой(2)
[33] вас(10)
[41] другим(2)
[44] безмолвно(2) -> еще(2) [collision]
[52] ови(1)
[60] совсем(2)
[61] вам(2)
[62] печаль(2)
[67] бог(2)
[69] так(4)
[70] нежно(2)
[74] она(2)
[77] тревожит(2)
[86] больше(2) -> душе(2) [collision]
[99] ничем(2)
[103] искренно(2)
[104] безнадежно(2)
[107] угасла(2)
[108] любил(6)
[115] не(6)
[122] робостью(2)
[124] томим(2) -> но(2) [collision]

```

Рис. 18. Содержимое хеш-таблицы с коллизиями

```

[126] быть(4)
Total unique words: 36

```

Рис. 19. Содержимое хеш-таблицы (окончание)

4.13 Статистика хеш-таблицы (пункт 13)

На рис. 20 показана статистика хеш-таблицы: размер 127, занято 32 корзины, 36 слов, коэффициент загрузки 0.28, всего 4 коллизии, максимальная длина цепочки 2.

```
Choice:13
13
=== Hash Table Statistics ===
Table size: 127
Used buckets: 32/127
Total unique words: 36
Load factor: 0.283465
Total collisions: 4
Max chain length: 2
Hash function: polynomial (base=31, mod=127)
```

Рис. 20. Статистика хеш-таблицы

4.14 Очистка хеш-таблицы (пункт 14)

На рис. 21 показана очистка хеш-таблицы.

```
Choice:14
14
Hash Table dictionary cleared.
```

Рис. 21. Очистка хеш-таблицы

4.15 Выход из программы (пункт 0)

На рис. 22 показан корректный выход из программы.

```
Choice:0
0
Goodbye!
```

Рис. 22. Выход из программы

Заключение

В результате работы реализованы два словаря для произвольного текста на русском языке:

1. Словарь на основе красно-чёрного дерева с функциями добавления, удаления, поиска, визуализации структуры дерева, полной очистки и загрузки из текстового файла.
2. Словарь на основе хеш-таблицы с методом цепочек и полиномиальной хеш-функцией (основание 31, модуль 127). Реализованы функции добавления, удаления, поиска, вывода статистики, полной очистки и загрузки из файла.

Все структуры данных реализованы без использования готовых контейнеров STL.

Программа корректно обрабатывает русский текст в кодировке UTF-8, приводит слова к нижнему регистру и валидирует пользовательский ввод.

Характеристики хеш-функции:

- Полиномиальная хеш-функция использует все символы слова (нетривиальная);
- Для 36 уникальных слов при размере таблицы 127 получено всего 4 коллизии;
- Коэффициент загрузки: 0.28;
- Максимальная длина цепочки: 2.

Корректность КЧ-дерева:

- Проверена последовательным удалением 5 узлов, включая корень;
- После каждого удаления дерево сохраняет все 5 свойств КЧ-дерева;
- Визуализация дерева до и после удаления подтверждает корректность перебалансировки;
- NIL-листья корректно отображаются как чёрные узлы.

Достоинства:

- Оба словаря работают над одним и тем же текстом и могут загружать данные из файла;
- Визуализация КЧ-дерева позволяет наглядно демонстрировать балансировку;
- Валидация ввода предотвращает добавление нерусских слов и некорректных команд.

Недостатки:

- Отсутствие графического интерфейса;
- Фиксированный размер хеш-таблицы (127 корзин).

Список использованной литературы

- [1] Хабр — красно-черные деревья. Текст: электронный — URL: <https://habr.com/ru/articles/330644/> (Дата обращения: 20.05.2026)
- [2] Кормен Т. и др. Алгоритмы: построение и анализ. — М.: Вильямс, 2013. — 1328 с.(дата обращения: 21.05.2025).
- [3] Секция «Телематика» / текст: электронный. — URL: <https://tema.spbstu.ru/tgraph/> (дата обращения: 28.05.2025).